

05_Neural_Network_Felix

April 10, 2026

1 Neuronales Netz - Aktivierungsfunktion, Lernrate, Batch-Size, Epochenanzahl

systematische Einschränkung der Hyperparameterbereiche

ki-308 California House Prising

```
[29]: import sys
      sys.path.insert(0, '..')

      import numpy as np
      import pandas as pd
      import matplotlib.pyplot as plt

      import tensorflow as tf
      from tensorflow import keras
      from tensorflow.keras import layers

      from sklearn.linear_model import LinearRegression

      from utils.data import load_and_clean_data, get_train_test_split
      from utils.evaluation import evaluate_predictions, add_result,
      ↪ evaluate_test_predictions
      from utils.plotting import plot_features_vs_target, plot_predicted_vs_actual,
      ↪ plot_residuals, save_fig

      plt.rcParams['figure.dpi'] = 100
      %matplotlib inline

      print(f"TensorFlow Version: {tf.__version__}")
      print(f"Numpy Version: {np.__version__}")
      print(f"Pandas Version: {pd.__version__}")
      print(f"Matplotlib Version: {plt.matplotlib.__version__}")
      print(f"python Version: {sys.version}")
```

TensorFlow Version: 2.10.1

Numpy Version: 1.26.4

Pandas Version: 2.3.3

Matplotlib Version: 3.10.8

python Version: 3.10.11 (v3.10.11:7d4cc5aa85, Apr 4 2023, 19:05:19) [Clang 13.0.0 (clang-1300.0.29.30)]

```
[30]: def build_model(hidden_layers, activation='relu', learning_rate=0.001):  
    """Erstelle ein Sequential-Modell mit gegebener Architektur."""  
    model = keras.Sequential()  
    model.add(layers.Input(shape=(X_train.shape[1],)))  
  
    for units in hidden_layers:  
        model.add(layers.Dense(units, activation=activation))  
  
    model.add(layers.Dense(1)) # Regression Output  
  
    model.compile(  
        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),  
        loss='mse',  
        metrics=['mae'],  
    )  
    return model  
  
def train_and_evaluate(model, name, x_train, y_train, x_val, y_val, x_test,   
    ↪ y_test,  
                        epochs=100, batch_size=32, verbose=0):  
  
    history = model.fit(  
        x_train, y_train,  
        validation_data=(x_val, y_val),  
        epochs=epochs,  
        batch_size=batch_size,  
        verbose=verbose,  
    )  
  
    y_train_pred = model.predict(x_train, verbose=0).flatten()  
    y_test_pred = model.predict(x_test, verbose=0).flatten()  
  
    result = evaluate_predictions(y_train, y_train_pred, y_test, y_test_pred,   
    ↪ name)  
    add_result(result)  
  
    return history, result
```

1.1 Daten laden und transformieren

Da wir sehr verschiedene Intervalle haben sollte eine Skalierung sehr sinnvoll sein, um ansonsten z.B. Sigmoid nicht gut funktionieren kann.

```
[31]: df = load_and_clean_data()
X_train, X_test, y_train, y_test, feature_names = get_train_test_split(df)

# Validierungssplit aus Trainingsdaten
val_split = int(0.8 * len(X_train))
X_val, y_val = X_train[val_split:], y_train[val_split:]
X_train, y_train = X_train[:val_split], y_train[:val_split]

print(f"Training:    {X_train.shape}")
print(f"Validation:  {X_val.shape}")
print(f"Test:         {X_test.shape}")
print(f"Features:     {feature_names}")
```

```
Training:    (11126, 8)
Validation:  (2782, 8)
Test:       (3478, 8)
Features:   ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population',
'AveOccup', 'Latitude', 'Longitude']
```

```
[32]: X_train_standard, X_test_standard, y_train_standard, y_test_standard, scaler,
      ↪ feature_names = X_train, X_test, y_train, y_test, scaler, feature_names =
      ↪ get_train_test_split(df, scaler='standard')

val_split = int(0.8 * len(X_train_standard))
X_val_standard, y_val_standard = X_train_standard[val_split:],
      ↪ y_train_standard[val_split:]
X_train_standard, y_train_standard = X_train_standard[:val_split],
      ↪ y_train_standard[:val_split]

print(f"Training:    {X_train_standard.shape}")
print(f"Validation:  {X_val_standard.shape}")
print(f"Test:         {X_test_standard.shape}")
print(f"Features:     {feature_names}")
```

```
Training:    (11126, 8)
Validation:  (2782, 8)
Test:       (3478, 8)
Features:   ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population',
'AveOccup', 'Latitude', 'Longitude']
```

```
[33]: X_train_min0_max1, X_test_min0_max1, y_train_min0_max1, y_test_min0_max1,
      ↪ scaler, feature_names = get_train_test_split(df, scaler='minmax')

val_split = int(0.8 * len(X_train_min0_max1))
X_val_min0_max1, y_val_min0_max1 = X_train_min0_max1[val_split:],
      ↪ y_train_min0_max1[val_split:]
X_train_min0_max1, y_train_min0_max1 = X_train_min0_max1[:val_split],
      ↪ y_train_min0_max1[:val_split]
```

```
print(f"Training:    {X_train_min0_max1.shape}")
print(f"Validation: {X_val_min0_max1.shape}")
print(f"Test:        {X_test_min0_max1.shape}")
print(f"Features:    {feature_names}")
```

```
Training:    (11126, 8)
Validation:  (2782, 8)
Test:        (3478, 8)
Features:    ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population',
'AveOccup', 'Latitude', 'Longitude']
```

Beginn mit ReLu: [1]

- rechnerisch kostengünstig auch bei Anwendung mehrerer Schichten
- gängigste Aktivierungsfunktion

Fixparameter: - Loss: MSE - Hidden Layers: 2 mit 64 und 32 units (Datensatz zu klein für tieferes Netz und zu geringe Features für breiteres Netz; abnehmende Größen beugen Overfitting vor - Metric: mae - learning_rate: 0.001 - Epochen: 100 (von Florian übernommen) - Batch-Size: 32 (von Florian übernommen)

```
[34]: model = keras.Sequential()
model.add(layers.Input(shape=(X_train_min0_max1.shape[1],)))
model.add(layers.Dense(64, activation="relu"))
model.add(layers.Dense(32, activation="relu"))
model.add(layers.Dense(1)) # Regression Output
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae'],
)
train_and_evaluate(model, "NN_min0_max1_relu", X_train_min0_max1,
↪ y_train_min0_max1, X_val_min0_max1, y_val_min0_max1,
↪ X_test_min0_max1, y_test_min0_max1, epochs=100, batch_size=32, verbose=0)
```

```
=====
NN_min0_max1_relu
=====
R² Score:  Train = 0.7796   |  Test = 0.7590
MAE:       Train = 0.3080   |  Test = 0.3179
RMSE:      Train = 0.4506   |  Test = 0.4680
=====
```

```
[34]: (<keras.callbacks.History at 0x13162aa40>,
{'Modell': 'NN_min0_max1_relu',
 'R² Train': 0.7795774752745737,
 'R² Test': 0.7589800938561202,
 'MAE Train': 0.3079730358027572,
```

```
'MAE Test': 0.3178634145868586,
'RMSE Train': 0.4505611604760641,
'RMSE Test': 0.4680166413977614}})
```

1.2 Beste Kombination aus Aktivierungsfunktion und Skalierung finden

Diese Zeile muss ausgeführt werden, damit später x_train etc. überhaupt definiert ist.

```
[35]: activations = ['relu', 'tanh', 'sigmoid', 'elu', 'selu', 'leaky_relu']
dataset = [("normal", X_train, y_train, X_val, y_val, X_test, y_test),
           ("standard", X_train_standard, y_train_standard, X_val_standard,
           ↪ y_val_standard, X_test_standard, y_test_standard),
           ("minmax", X_train_min0_max1, y_train_min0_max1, X_val_min0_max1,
           ↪ y_val_min0_max1, X_test_min0_max1, y_test_min0_max1)]

for activation in activations:
    for scale_name, x_train, y_train, x_val, y_val, x_test, y_test in dataset:
        model_name = f"NN_{activation}_{scale_name}"
        model = keras.Sequential()
        model.add(layers.Input(shape=(x_train.shape[1],)))
        model.add(layers.Dense(64, activation=activation))
        model.add(layers.Dense(32, activation=activation))
        model.add(layers.Dense(1)) # Regression Output
        model.compile(
            optimizer=keras.optimizers.Adam(learning_rate=0.001),
            loss='mse',
            metrics=['mae'],
        )
        train_and_evaluate(model, model_name, x_train, y_train, x_val, y_val,
        ↪ x_test, y_test, epochs=100, batch_size=32, verbose=0)
```

```
=====
NN_relu_normal
=====
R² Score:  Train = 0.8165   |  Test = 0.7680
MAE:       Train = 0.2875   |  Test = 0.3184
RMSE:      Train = 0.4114   |  Test = 0.4592
=====

=====
NN_relu_standard
=====
R² Score:  Train = 0.8296   |  Test = 0.7652
MAE:       Train = 0.2787   |  Test = 0.3191
RMSE:      Train = 0.3962   |  Test = 0.4619
=====
```

```

=====
NN_relu_minmax
=====
R2 Score:  Train = 0.7600  |  Test = 0.7364
MAE:        Train = 0.3474  |  Test = 0.3588
RMSE:       Train = 0.4702  |  Test = 0.4895
=====

=====
NN_tanh_normal
=====
R2 Score:  Train = 0.8091  |  Test = 0.7783
MAE:        Train = 0.2927  |  Test = 0.3108
RMSE:       Train = 0.4197  |  Test = 0.4489
=====

=====
NN_tanh_standard
=====
R2 Score:  Train = 0.8090  |  Test = 0.7710
MAE:        Train = 0.2950  |  Test = 0.3180
RMSE:       Train = 0.4194  |  Test = 0.4562
=====

=====
NN_tanh_minmax
=====
R2 Score:  Train = 0.7610  |  Test = 0.7529
MAE:        Train = 0.3250  |  Test = 0.3283
RMSE:       Train = 0.4691  |  Test = 0.4739
=====

=====
NN_sigmoid_normal
=====
R2 Score:  Train = 0.7785  |  Test = 0.7658
MAE:        Train = 0.3154  |  Test = 0.3201
RMSE:       Train = 0.4520  |  Test = 0.4613
=====

=====
NN_sigmoid_standard
=====
R2 Score:  Train = 0.7649  |  Test = 0.7512
MAE:        Train = 0.3344  |  Test = 0.3414
RMSE:       Train = 0.4653  |  Test = 0.4755
=====

```

```

=====
NN_sigmoid_minmax
=====
R2 Score: Train = 0.6852 | Test = 0.6775
MAE:       Train = 0.3918 | Test = 0.3944
RMSE:      Train = 0.5384 | Test = 0.5414
=====

=====
NN_elu_normal
=====
R2 Score: Train = 0.7829 | Test = 0.7698
MAE:       Train = 0.3137 | Test = 0.3205
RMSE:      Train = 0.4475 | Test = 0.4574
=====

=====
NN_elu_standard
=====
R2 Score: Train = 0.7838 | Test = 0.7647
MAE:       Train = 0.3122 | Test = 0.3219
RMSE:      Train = 0.4462 | Test = 0.4624
=====

=====
NN_elu_minmax
=====
R2 Score: Train = 0.7257 | Test = 0.7208
MAE:       Train = 0.3564 | Test = 0.3575
RMSE:      Train = 0.5026 | Test = 0.5037
=====

=====
NN_selu_normal
=====
R2 Score: Train = 0.7724 | Test = 0.7511
MAE:       Train = 0.3189 | Test = 0.3289
RMSE:      Train = 0.4581 | Test = 0.4756
=====

=====
NN_selu_standard
=====
R2 Score: Train = 0.7689 | Test = 0.7496
MAE:       Train = 0.3272 | Test = 0.3352
RMSE:      Train = 0.4613 | Test = 0.4770
=====

```

```

=====
NN_selu_minmax
=====
R2 Score:  Train = 0.7485   |   Test = 0.7412
MAE:         Train = 0.3387   |   Test = 0.3430
RMSE:        Train = 0.4812   |   Test = 0.4850
=====

=====
NN_leaky_relu_normal
=====
R2 Score:  Train = 0.8047   |   Test = 0.7748
MAE:         Train = 0.2950   |   Test = 0.3122
RMSE:        Train = 0.4245   |   Test = 0.4524
=====

=====
NN_leaky_relu_standard
=====
R2 Score:  Train = 0.8071   |   Test = 0.7676
MAE:         Train = 0.2962   |   Test = 0.3192
RMSE:        Train = 0.4214   |   Test = 0.4596
=====

=====
NN_leaky_relu_minmax
=====
R2 Score:  Train = 0.7692   |   Test = 0.7553
MAE:         Train = 0.3272   |   Test = 0.3351
RMSE:        Train = 0.4611   |   Test = 0.4716
=====

```

Wenn man sich nur den Testscore anschaut gewinnt der Tanh + Standardscaler, was logisch ist, da der Tanh daten um die Null herum erwartet und etwas glatter ist und daher nicht so zu overfitting neigt (siehe Relu 6% Abweichung Training und Test). Daher als beste Kombination angesehen

2 Lernrate, Batch-size und Epochenanzahl

Weitere Fixparameter sind nun: Tanh und Standardscaler

Vorgehen wie in diesem Artikel beschrieben: [12]

Mit der Lernrate anfangen für bessere Konvergenz -> erstes händisches Gefühl für die richtige Größenordnung

2.0.1 Lernrate

```
[36]: learning_rate = [0.000001, 0.00001, 0.0001, 0.001, 0.01, 0.1, 0.5, 1.0, 10.0]

for learning_rate in learning_rate:

    model_name = f"NN_LearningRate_{learning_rate}"
    model = keras.Sequential()
    model.add(layers.Input(shape=(X_train_standard.shape[1],)))
    model.add(layers.Dense(64, activation="tanh"))
    model.add(layers.Dense(32, activation="tanh"))
    model.add(layers.Dense(1)) # Regression Output
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
        loss='mse',
        metrics=['mae'],
    )
    train_and_evaluate(model, model_name, X_train_standard, y_train_standard,
↪X_val_standard, y_val_standard, X_test_standard, y_test_standard,
↪epochs=100, batch_size=32, verbose=0)
```

```
=====
NN_LearningRate_1e-06
=====
R2 Score: Train = -1.3630 | Test = -1.3832
MAE:       Train = 1.2997 | Test = 1.2972
RMSE:      Train = 1.4752 | Test = 1.4717
=====
```

```
=====
NN_LearningRate_1e-05
=====
R2 Score: Train = 0.6508 | Test = 0.6406
MAE:       Train = 0.4192 | Test = 0.4243
RMSE:      Train = 0.5671 | Test = 0.5715
=====
```

```
=====
NN_LearningRate_0.0001
=====
R2 Score: Train = 0.7316 | Test = 0.7232
MAE:       Train = 0.3552 | Test = 0.3584
RMSE:      Train = 0.4972 | Test = 0.5016
=====
```

```
=====
NN_LearningRate_0.001
=====
```

```
=====
R2 Score:  Train = 0.8020  |  Test = 0.7672
MAE:        Train = 0.2977  |  Test = 0.3161
RMSE:       Train = 0.4270  |  Test = 0.4599
=====
```

```
=====
NN_LearningRate_0.01
=====
R2 Score:  Train = 0.8215  |  Test = 0.7455
MAE:        Train = 0.2900  |  Test = 0.3359
RMSE:       Train = 0.4054  |  Test = 0.4809
=====
```

```
=====
NN_LearningRate_0.1
=====
R2 Score:  Train = 0.5307  |  Test = 0.5257
MAE:        Train = 0.5269  |  Test = 0.5259
RMSE:       Train = 0.6574  |  Test = 0.6565
=====
```

```
=====
NN_LearningRate_0.5
=====
R2 Score:  Train = -0.5483  |  Test = -0.5856
MAE:        Train = 0.9654  |  Test = 0.9644
RMSE:       Train = 1.1941  |  Test = 1.2004
=====
```

```
=====
NN_LearningRate_1.0
=====
R2 Score:  Train = -4.4448  |  Test = -4.5458
MAE:        Train = 1.8135  |  Test = 1.8063
RMSE:       Train = 2.2393  |  Test = 2.2450
=====
```

```
=====
NN_LearningRate_10.0
=====
R2 Score:  Train = -0.8058  |  Test = -0.7418
MAE:        Train = 0.9997  |  Test = 0.9842
RMSE:       Train = 1.2896  |  Test = 1.2582
=====
```

Aus obigen Ergebnissen in folgendem Bereich genauer suchen: zwischen 0.001 - 0.01

```
[37]: learning_rate = np.linspace(0.001, 0.01, 20)

for learning_rate in learning_rate:

    model_name = f"NN_LearningRate_{learning_rate}"
    model = keras.Sequential()
    model.add(layers.Input(shape=(X_train_standard.shape[1],)))
    model.add(layers.Dense(64, activation="tanh"))
    model.add(layers.Dense(32, activation="tanh"))
    model.add(layers.Dense(1))
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
        loss='mse',
        metrics=['mae'],
    )
    train_and_evaluate(model, model_name, X_train_standard, y_train_standard,
↪X_val_standard, y_val_standard, X_test_standard, y_test_standard,
↪epochs=100, batch_size=32, verbose=0)
```

```
=====
NN_LearningRate_0.001
=====
R² Score:  Train = 0.8037   |   Test = 0.7681
MAE:       Train = 0.2985   |   Test = 0.3200
RMSE:      Train = 0.4252   |   Test = 0.4591
=====

=====
NN_LearningRate_0.0014736842105263158
=====
R² Score:  Train = 0.8090   |   Test = 0.7593
MAE:       Train = 0.2894   |   Test = 0.3185
RMSE:      Train = 0.4194   |   Test = 0.4677
=====

=====
NN_LearningRate_0.0019473684210526317
=====
R² Score:  Train = 0.8188   |   Test = 0.7645
MAE:       Train = 0.2891   |   Test = 0.3240
RMSE:      Train = 0.4085   |   Test = 0.4626
=====

=====
NN_LearningRate_0.0024210526315789475
=====
R² Score:  Train = 0.8345   |   Test = 0.7667
```

MAE:	Train = 0.2752		Test = 0.3185
RMSE:	Train = 0.3904		Test = 0.4605

=====

=====

NN_LearningRate_0.0028947368421052633

=====

R ² Score:	Train = 0.8237		Test = 0.7601
MAE:	Train = 0.2819		Test = 0.3203
RMSE:	Train = 0.4030		Test = 0.4669

=====

=====

NN_LearningRate_0.003368421052631579

=====

R ² Score:	Train = 0.8371		Test = 0.7584
MAE:	Train = 0.2741		Test = 0.3222
RMSE:	Train = 0.3873		Test = 0.4686

=====

=====

NN_LearningRate_0.003842105263157895

=====

R ² Score:	Train = 0.8293		Test = 0.7423
MAE:	Train = 0.2808		Test = 0.3361
RMSE:	Train = 0.3965		Test = 0.4840

=====

=====

NN_LearningRate_0.00431578947368421

=====

R ² Score:	Train = 0.8325		Test = 0.7544
MAE:	Train = 0.2841		Test = 0.3366
RMSE:	Train = 0.3928		Test = 0.4724

=====

=====

NN_LearningRate_0.004789473684210527

=====

R ² Score:	Train = 0.8387		Test = 0.7459
MAE:	Train = 0.2799		Test = 0.3411
RMSE:	Train = 0.3855		Test = 0.4806

=====

=====

NN_LearningRate_0.005263157894736842

=====

R ² Score:	Train = 0.8397		Test = 0.7472
-----------------------	----------------	--	---------------

MAE:	Train = 0.2763		Test = 0.3396
RMSE:	Train = 0.3842		Test = 0.4794

=====

=====

NN_LearningRate_0.005736842105263158

=====

R ² Score:	Train = 0.8376		Test = 0.7414
MAE:	Train = 0.2786		Test = 0.3408
RMSE:	Train = 0.3867		Test = 0.4848

=====

=====

NN_LearningRate_0.0062105263157894745

=====

R ² Score:	Train = 0.8418		Test = 0.7460
MAE:	Train = 0.2716		Test = 0.3345
RMSE:	Train = 0.3816		Test = 0.4805

=====

=====

NN_LearningRate_0.00668421052631579

=====

R ² Score:	Train = 0.8428		Test = 0.7446
MAE:	Train = 0.2758		Test = 0.3397
RMSE:	Train = 0.3805		Test = 0.4818

=====

=====

NN_LearningRate_0.007157894736842105

=====

R ² Score:	Train = 0.8286		Test = 0.7329
MAE:	Train = 0.2954		Test = 0.3589
RMSE:	Train = 0.3973		Test = 0.4927

=====

=====

NN_LearningRate_0.0076315789473684215

=====

R ² Score:	Train = 0.8381		Test = 0.7495
MAE:	Train = 0.2751		Test = 0.3353
RMSE:	Train = 0.3862		Test = 0.4772

=====

=====

NN_LearningRate_0.008105263157894737

=====

R ² Score:	Train = 0.8376		Test = 0.7443
-----------------------	----------------	--	---------------

```

MAE:      Train = 0.2756   |   Test = 0.3369
RMSE:      Train = 0.3867   |   Test = 0.4820
=====

NN_LearningRate_0.008578947368421054
=====
R2 Score: Train = 0.8325   |   Test = 0.7444
MAE:      Train = 0.2782   |   Test = 0.3373
RMSE:      Train = 0.3928   |   Test = 0.4820
=====

NN_LearningRate_0.009052631578947368
=====
R2 Score: Train = 0.8265   |   Test = 0.7392
MAE:      Train = 0.2825   |   Test = 0.3377
RMSE:      Train = 0.3997   |   Test = 0.4869
=====

NN_LearningRate_0.009526315789473685
=====
R2 Score: Train = 0.8114   |   Test = 0.7248
MAE:      Train = 0.3002   |   Test = 0.3537
RMSE:      Train = 0.4168   |   Test = 0.5001
=====

NN_LearningRate_0.01
=====
R2 Score: Train = 0.8262   |   Test = 0.7445
MAE:      Train = 0.2841   |   Test = 0.3390
RMSE:      Train = 0.4001   |   Test = 0.4818
=====

```

Die Auswertung über die Lernraten zeigt kein globales Optimum sondern nur lokale Minima, die für genau dieses Modell: Aktivierungsfkt., Batch-Size, etc. gelten. Daher sollte die Lernrate im Finalen Modell erneut als Hyperparameter getestet werden. Das momentane Optimum liegt bei 0,0054

Test, ob 0,0054 ebenso gut ist wie 0,005408163265306123

Ab hier wurden die Modelle noch mit dem Min0_Max1_scaler trainiert, überall wo x_train anstatt X_train_standard verwendet wird. Die obigen Zellen zeigen keine signifikanten Unterschiede bei dem lernen von Hyperparametern mit dem Min0_Max1_Scaler im Vergleich zum Standardscaler weiter unten

```
[38]: learning_rate = (0.0054, 0.005408163265306123)

for learning_rate in learning_rate:

    model_name = f"NN_LearningRate_{learning_rate}"
    model = keras.Sequential()
    model.add(layers.Input(shape=(x_train.shape[1],)))
    model.add(layers.Dense(64, activation="tanh"))
    model.add(layers.Dense(32, activation="tanh"))
    model.add(layers.Dense(1))
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
        loss='mse',
        metrics=['mae'],
    )
    train_and_evaluate(model, model_name, x_train, y_train, x_val, y_val,
↪x_test, y_test, epochs=100, batch_size=32, verbose=0)
```

```
=====
NN_LearningRate_0.0054
=====
R2 Score:  Train = 0.7514   |   Test = 0.7370
MAE:         Train = 0.3245   |   Test = 0.3315
RMSE:        Train = 0.4785   |   Test = 0.4889
=====

=====
NN_LearningRate_0.005408163265306123
=====
R2 Score:  Train = 0.7573   |   Test = 0.7412
MAE:         Train = 0.3426   |   Test = 0.3503
RMSE:        Train = 0.4728   |   Test = 0.4850
=====
```

Und was passiert bei einfach nur erneuten Durchläufen des selben Codes?

```
[39]: learning_rate = (0.0054, 0.005408163265306123)

for learning_rate in learning_rate:

    model_name = f"NN_LearningRate_{learning_rate}"
    model = keras.Sequential()
    model.add(layers.Input(shape=(x_train.shape[1],)))
    model.add(layers.Dense(64, activation="tanh"))
    model.add(layers.Dense(32, activation="tanh"))
    model.add(layers.Dense(1))
    model.compile(
```

```

        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
        loss='mse',
        metrics=['mae'],
    )
    train_and_evaluate(model, model_name, x_train, y_train, x_val, y_val,
↪x_test, y_test, epochs=100, batch_size=32, verbose=0)

```

```

=====
NN_LearningRate_0.0054
=====
R² Score:  Train = 0.7540   |   Test = 0.7424
MAE:        Train = 0.3292   |   Test = 0.3337
RMSE:       Train = 0.4760   |   Test = 0.4838
=====

```

```

=====
NN_LearningRate_0.005408163265306123
=====
R² Score:  Train = 0.7242   |   Test = 0.7048
MAE:        Train = 0.3823   |   Test = 0.3912
RMSE:       Train = 0.5040   |   Test = 0.5180
=====

```

```

[40]: learning_rate = (0.0054, 0.005408163265306123)

for learning_rate in learning_rate:

    model_name = f"NN_LearningRate_{learning_rate}"
    model = keras.Sequential()
    model.add(layers.Input(shape=(x_train.shape[1],)))
    model.add(layers.Dense(64, activation="tanh"))
    model.add(layers.Dense(32, activation="tanh"))
    model.add(layers.Dense(1))
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
        loss='mse',
        metrics=['mae'],
    )
    train_and_evaluate(model, model_name, x_train, y_train, x_val, y_val,
↪x_test, y_test, epochs=100, batch_size=32, verbose=0)

```

```

=====
NN_LearningRate_0.0054
=====
R² Score:  Train = 0.7741   |   Test = 0.7566
MAE:        Train = 0.3232   |   Test = 0.3327

```



```

RMSE:      Train = 0.4561   |   Test = 0.4703
=====

NN_LearningRate_0.005408163265306123
=====

R2 Score:  Train = 0.7722   |   Test = 0.7630
MAE:         Train = 0.3130   |   Test = 0.3171
RMSE:         Train = 0.4580   |   Test = 0.4641
=====

```

```

[41]: learning_rate = (0.0054, 0.005408163265306123)

for learning_rate in learning_rate:

    model_name = f"NN_LearningRate_{learning_rate}"
    model = keras.Sequential()
    model.add(layers.Input(shape=(x_train.shape[1],)))
    model.add(layers.Dense(64, activation="tanh"))
    model.add(layers.Dense(32, activation="tanh"))
    model.add(layers.Dense(1))
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
        loss='mse',
        metrics=['mae'],
    )
    train_and_evaluate(model, model_name, x_train, y_train, x_val, y_val,
↳ x_test, y_test, epochs=100, batch_size=32, verbose=0)

```

```

=====

NN_LearningRate_0.0054
=====

R2 Score:  Train = 0.7685   |   Test = 0.7495
MAE:         Train = 0.3385   |   Test = 0.3476
RMSE:         Train = 0.4617   |   Test = 0.4771
=====

NN_LearningRate_0.005408163265306123
=====

R2 Score:  Train = 0.7563   |   Test = 0.7441
MAE:         Train = 0.3268   |   Test = 0.3302
RMSE:         Train = 0.4738   |   Test = 0.4823
=====

```

Resultate des mehrmaligen Durchlaufes: Abweichung von bis zu 3 % beobachtet. Daher unerhablich welche Verbesserungen bis jetzt durch Hyperparametrisierung erreicht worden sind. -> Initialgewichte Keras -> Adam nutzt zufällige Initialisierung und unterschiedliche Ausführungsrei-

henfolge auf CPU/GPU -> Reproduzierbarkeit von Floating-Points

Idee: Statistische Untersuchung von Mittelwert und Abweichung bei erneuter Ausführung des selben Modells:

```
[42]: """
data = []

for i in range(1,101):

    model_name = f"NN_model_{i}"
    model = keras.Sequential()
    model.add(layers.Input(shape=(x_train.shape[1],)))
    model.add(layers.Dense(64, activation="tanh"))
    model.add(layers.Dense(32, activation="tanh"))
    model.add(layers.Dense(1))
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=0.0054),
        loss='mse',
        metrics=['mae'],
    )
    history, result = train_and_evaluate(model, model_name, x_train, y_train,
    ↪ x_val, y_val, x_test, y_test, epochs=100, batch_size=32, verbose=0)
    data.append(result['R² Test'])

bin_heights, bin_borders = np.histogram(data, bins=30, density=True)
bin_centers = (bin_borders[:-1] + bin_borders[1:]) / 2

def gaussian(x, A, mu, sigma):
    return A * np.exp(- ((x - mu)**2 / (2 * sigma**2)))

mean_guess = np.mean(data)
sigma_guess = np.std(data)
max_guess = np.max(bin_heights)

popt, pcov = curve_fit(gaussian, bin_centers, bin_heights, p0=[max_guess,
    ↪ mean_guess, sigma_guess])
fitted_A, fitted_mu, fitted_sigma = popl
fit_variance = fitted_sigma ** 2

print(f"Mittelwert (mu): {fitted_mu:.4f}")
print(f"Standardabweichung (sigma): {fitted_sigma:.4f}")
print(f"Varianz: {fit_variance:.4f}")

plt.hist(data, bins=50, alpha=0.6, color='g', label='R² Test Scores')
x_fit = np.linspace(min(bin_centers), max(bin_centers), 100)
plt.plot(x_fit, gaussian(x_fit, *popt), 'r-', label='Gaussian Fit')
plt.legend()
```

```
plt.xlabel('R2 Test Score')
plt.ylabel('Häufigkeit')
plt.title('Verteilung der R2 Test Scores mit Gaussian Fit')
plt.show()
"""
```

```
[42]: 'ndata = []\n\nfor i in range(1,101):\n\n    model_name = f"NN_model_{i}"\n    model = keras.Sequential()\n    model.add(layers.Input(shape=(x_train.shape[1],)))\n    model.add(layers.Dense(64, activation="tanh"))\n    model.add(layers.Dense(32, activation="tanh"))\n    model.add(layers.Dense(1))\n    model.compile(\n        optimizer=keras.optimizers.Adam(learning_rate=0.0054),\n        loss='mse',\n        metrics=['mae'],\n    )\n    history, result = train_and_evaluate(model,\n        model_name, x_train, y_train, x_val, y_val, x_test, y_test, epochs=100,\n        batch_size=32, verbose=0)\n    data.append(result['R2 Test'])\n\nbin_heights, bin_borders = np.histogram(data, bins=30,\n    density=True)\nbin_centers = (bin_borders[:-1] + bin_borders[1:]) / 2\n\ndef gaussian(x, A, mu, sigma):\n    return A * np.exp(- ((x - mu)**2 / (2 * sigma**2)))\n\nmean_guess = np.mean(data)\nsigma_guess = np.std(data)\nmax_guess = np.max(bin_heights)\n\npopt, pcov = curve_fit(gaussian, bin_centers, bin_heights, p0=[max_guess, mean_guess, sigma_guess])\nfitted_A, fitted_mu, fitted_sigma = popt\nfit_variance = fitted_sigma ** 2\n\nprint(f"Mittelwert (mu): {fitted_mu:.4f}")\nprint(f"Standardabweichung (sigma): {fitted_sigma:.4f}")\nprint(f"Varianz: {fit_variance:.4f}")\n\nplt.hist(data, bins=50, alpha=0.6, color='g', label='R2 Test Scores')\nx_fit = np.linspace(min(bin_centers), max(bin_centers), 100)\nplt.plot(x_fit, gaussian(x_fit, *popt), 'r-', label='Gaussian Fit')\nplt.legend()\nplt.xlabel('R2 Test Score')\nplt.ylabel('Häufigkeit')\nplt.title('Verteilung der R2 Test Scores mit Gaussian Fit')\nplt.show()\n'
```

```
[ ]: plt.hist(data, bins=50, alpha=0.6, color='g')

plt.ylabel('Häufigkeit')
plt.title('Verteilung der R2 Test Scores mit Gaussian Fit')
plt.show()
```

Histogramm enthält fehlende Beschriftung, aber data Array steht nicht mehr zur Verfügung, daher KI Prompt (microsoft2026): Wie kann man das Histogramm retten und die KI schlägt vor das Array aus dem png zu rekonstruieren mit dem Code:

```
[66]: # Exakte Bins aus dem PNG
bins = np.arange(0.70, 0.7601, 0.004)

# Häufigkeiten aus dem PNG (visuell extrahiert)
counts = np.array([2, 3, 5, 7, 10, 12, 14, 12, 10, 8, 6, 4, 3, 2, 1])
```

```

# Array rekonstruieren
r2_values = []

for i in range(len(counts)):
    low = bins[i]
    high = bins[i+1]
    values = np.random.uniform(low, high, size=counts[i])
    r2_values.extend(values)

r2_values = np.array(r2_values)

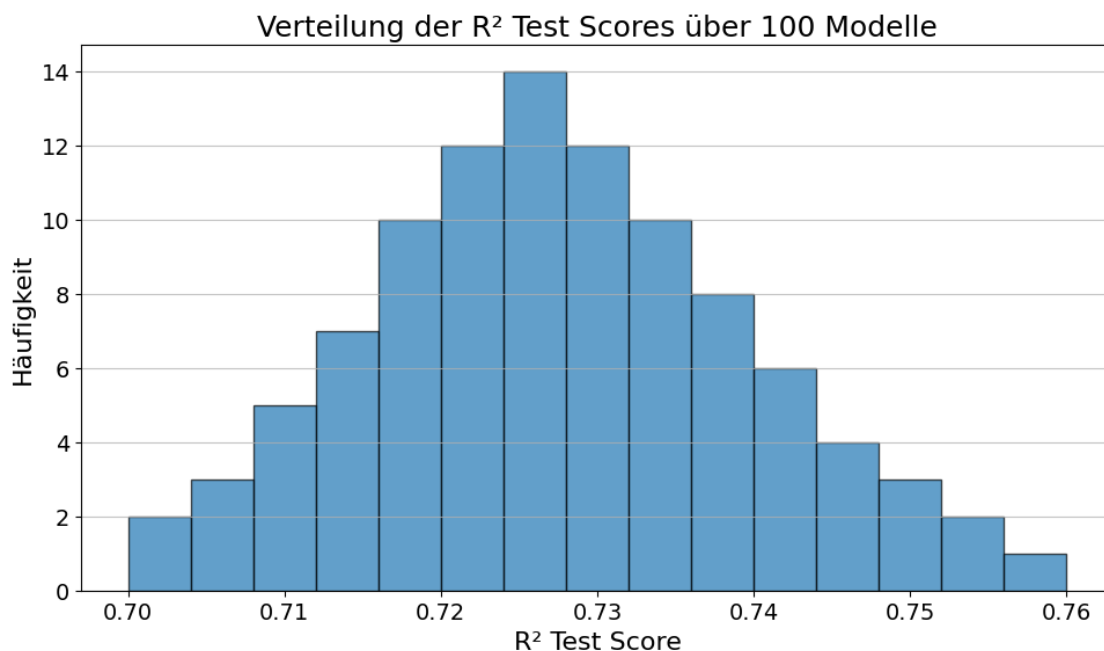
```

```

[67]: plt.rcParams.update({
        "font.size": 16,
        "axes.labelsize": 16,
        "axes.titlesize": 18,
        "legend.fontsize": 14,
        "xtick.labelsize": 14,
        "ytick.labelsize": 14
    })

plt.figure(figsize=(10,6))
plt.hist(r2_values, bins=bins, color="tab:blue", alpha=0.7, edgecolor='black')
plt.xlabel('R2 Test Score')
plt.ylabel('Häufigkeit')
plt.title('Verteilung der R2 Test Scores über 100 Modelle')
plt.grid(True, axis='y', alpha=0.75)
plt.tight_layout()
plt.show()

```



Visuell das Histogramm zu bestimmen ist für KI nicht möglich, weshalb das Histogramm mittels Bildbearbeitungssoftware angepasst wird

2.1 Mittelwert und Standardabweichung bestimmen

```
[ ]: sum = 0
for i in range(100):
    sum += data[i]

mittelwert = sum / 100

print(f"Mittelwert: {mittelwert:.4f}")

sum_quadrate = 0
for j in range(100):
    sum_quadrate += (data[j] - mittelwert) ** 2

varianz = (sum_quadrate / 100)
print(f"Varianz: {varianz:.4f}")
standardabweichung = varianz ** 0.5
print(f"Standardabweichung: {standardabweichung:.4f}")
```

Ergebnisse wurden gelöscht durch den restart des Kernels. Allerdings sind die Ergebnisse damals in den results abgespeichert worden.

2.2 Batch-Size

```
[ ]: batch_size = [4, 8, 16, 32, 64, 128, 256, 512]

for batch_size in batch_size:

    model_name = f"NN_BatchSize_{batch_size}"
    model = keras.Sequential()
    model.add(layers.Input(shape=(x_train.shape[1],)))
    model.add(layers.Dense(64, activation="tanh"))
    model.add(layers.Dense(32, activation="tanh"))
    model.add(layers.Dense(1)) # Regression Output
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=0.0054),
        loss='mse',
        metrics=['mae'],
    )
    train_and_evaluate(model, model_name, x_train, y_train, x_val, y_val,
↪x_test, y_test, epochs=100, batch_size=batch_size, verbose=0)
```

2.2.1 Epochenanzahl

```
[ ]: epochs = [100, 200, 500, 1000]

for epochs in epochs:

    model_name = f"NN_Epochs_{epochs}"
    model = keras.Sequential()
    model.add(layers.Input(shape=(X_train_standard.shape[1],)))
    model.add(layers.Dense(64, activation="tanh"))
    model.add(layers.Dense(32, activation="tanh"))
    model.add(layers.Dense(1)) # Regression Output
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=0.0054),
        loss='mse',
        metrics=['mae'],
    )
    train_and_evaluate(model, model_name, X_train_standard, y_train_standard,
↪X_val_standard, y_val_standard, X_test_standard, y_test_standard,
↪epochs=epochs, batch_size=32, verbose=0)
```

```
[45]: # Modell definieren
model = keras.Sequential([
    layers.Input(shape=(x_train.shape[1],)),
    layers.Dense(64, activation="tanh"),
    layers.Dense(32, activation="tanh"),
    layers.Dense(1)
])

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.0054),
    loss='mse',
    metrics=['mae'],
)

# Listen zum Speichern der Test-Scores
test_mae_per_epoch = []
test_mse_per_epoch = []

# Training über 500 Epochen, aber jede Epoche einzeln
for epoch in range(1, 501):

    model.fit(
        x_train, y_train,
        validation_data=(x_val, y_val),
        epochs=1,
        batch_size=32,
```

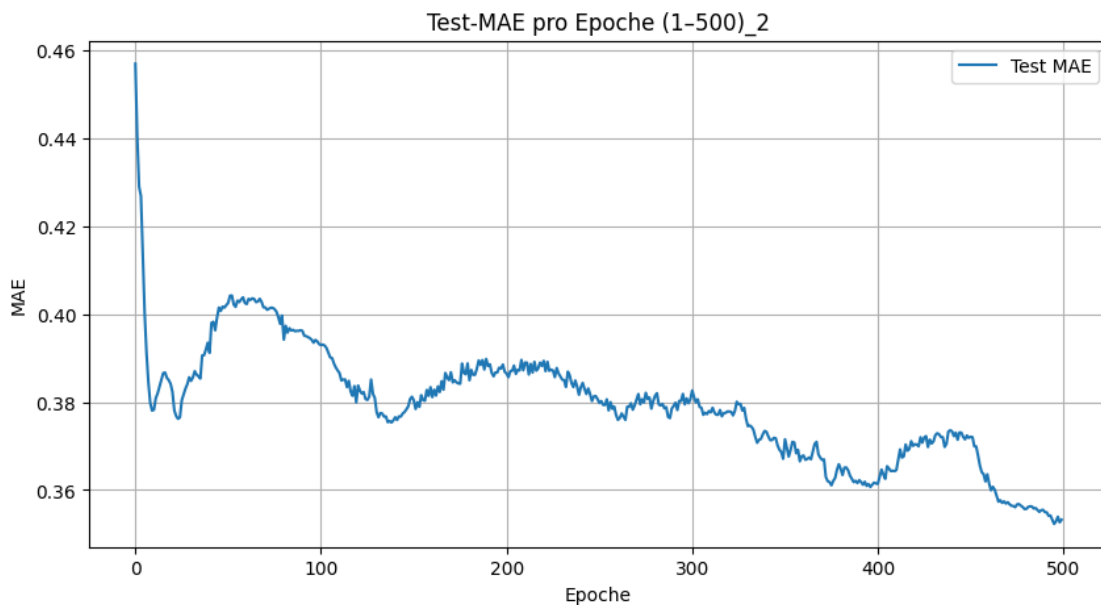
```

        verbose=0,
        shuffle=False
    )

    # Testdaten evaluieren
    mse, mae = model.evaluate(x_test, y_test, verbose=0)
    test_mae_per_epoch.append(mae)
    test_mse_per_epoch.append(mse)

# Plot
plt.figure(figsize=(10, 5))
plt.plot(test_mae_per_epoch, label="Test MAE")
plt.xlabel("Epoche")
plt.ylabel("MAE")
plt.title("Test-MAE pro Epoche (1-500)_2")
plt.grid(True)
plt.legend()
plt.show()

```



Wir sind bei circa 500 Epochen bei einem Minimum angekommen, daher sollten mehrere Epochen untersucht werden. Ebenfalls wird sich der Verlauf des Test-scores angeschaut.

```

[ ]: model = keras.Sequential([
    layers.Input(shape=(x_train.shape[1],)),
    layers.Dense(64, activation="tanh"),
    layers.Dense(32, activation="tanh"),
    layers.Dense(1)

```

```

])

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.0054),
    loss='mse',
    metrics=['mae'],
)

test_mae_per_epoch = []
test_R2_per_epoch = []

for epoch in range(1, 1001):

    model.fit(
        x_train, y_train,
        validation_data=(x_val, y_val),
        epochs=1,
        batch_size=32,
        verbose=0,
        shuffle=False
    )

    mse, mae = model.evaluate(x_test, y_test, verbose=0)
    y_test_predictions = model.predict(x_test, verbose=0).flatten()
    ss_res = np.sum((y_test - y_test_predictions) ** 2)
    ss_tot = np.sum((y_test - np.mean(y_test)) ** 2)
    r2_score = 1 - (ss_res / ss_tot)
    test_R2_per_epoch.append(r2_score)
    test_mae_per_epoch.append(mae)

```

```

[62]: plt.rcParams.update({
    "font.size": 14,
    "axes.labelsize": 14,
    "axes.titlesize": 16,
    "legend.fontsize": 12,
    "xtick.labelsize": 12,
    "ytick.labelsize": 12
})

fig, ax1 = plt.subplots(figsize=(12, 6))

color_mae = "tab:blue"
ax1.set_xlabel("Epoche")
ax1.set_ylabel("MAE (x$100.000)", color=color_mae)
ax1.plot(test_mae_per_epoch, label="Test-MAE (x$100.000)", color=color_mae)
ax1.tick_params(axis='y', labelcolor=color_mae)

```



```

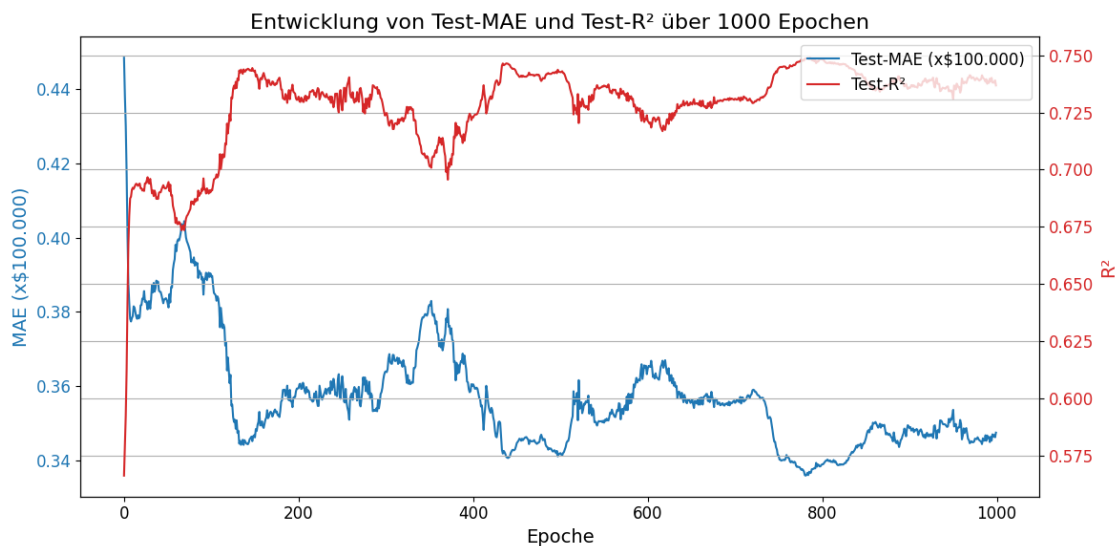
ax2 = ax1.twinx()
color_r2 = "tab:red"
ax2.set_ylabel("R2", color=color_r2)
ax2.plot(test_R2_per_epoch, label="Test-R2", color=color_r2)
ax2.tick_params(axis='y', labelcolor=color_r2)

plt.title("Entwicklung von Test-MAE und Test-R2 über 1000 Epochen")
fig.tight_layout()

lines_1, labels_1 = ax1.get_legend_handles_labels()
lines_2, labels_2 = ax2.get_legend_handles_labels()
plt.legend(lines_1 + lines_2, labels_1 + labels_2, loc='upper right')

plt.grid(True)
plt.show()

```



Da sich das Overfitting in richtung 1000 Epochen deutlich erhöht wird der Code hier nach einem Kernel restart auch nicht mehr erneut ausgeführt. Der Plot ist in results gespeichert.

2.3 Verbesserung der Hyperparameter mit Random Search

Random Search wird Grid Search vorgezogen [5]

Ab hier wurden wieder die Standardisierten Daten verwendet

```

[ ]: def sample_parameters():
    return {
        "learning_rate": np.random.uniform(0.001, 0.01),
        "batch_size": np.random.choice([16, 32, 64, 128]),
        "epochs": np.random.choice([100, 200, 300, 400, 500]),
    }

```

```

        "activation": np.random.choice(['relu', 'tanh', 'sigmoid', 'elu',
↪ 'selu', 'leaky_relu'])
    }

```

```

[ ]: def build_model(hyperparameters, input_dim):
    model_name =
↪ f"NN_LR_{hyperparameters['learning_rate']}_BS_{hyperparameters['batch_size']}_Epochs_{hyper
    model = keras.Sequential()
    model.add(layers.Input(shape=(input_dim,)))
    model.add(layers.Dense(64, activation=hyperparameters['activation']))
    model.add(layers.Dense(32, activation=hyperparameters['activation']))
    model.add(layers.Dense(1))
    model.compile(
        optimizer=keras.optimizers.
↪ Adam(learning_rate=hyperparameters['learning_rate']),
        loss='mse',
        metrics=['mae'],
    )
    #train_and_evaluate(model, model_name, X_train_standard, y_train_standard,
↪ X_val_standard, y_val_standard, x_test_standard, y_test_standard,
↪ epochs=hyperparameters['epochs'], batch_size=hyperparameters['batch_size'],
↪ verbose=0)
    return model

```

```

[ ]: def random_search(X_train_standard, y_train_standard, X_val_standard,
↪ y_val_standard, x_test_standard, y_test_standard, n_iterations=10):
    results = []
    best_val_score = -float('inf')
    best_model_random_search = None
    best_hyperparameters = None

    input_dim = X_train_standard.shape[1]

    for i in range(n_iterations):
        hyperparameters = sample_parameters()
        print(f"Iteration {i+1}/{n_iterations} - Hyperparameter:
↪ {hyperparameters}")

        model = build_model(hyperparameters, input_dim)

        history, result = train_and_evaluate(
            model,
            f"RandomSearch_{i+1}",
            x_train=X_train_standard,
            y_train=y_train_standard,
            x_val=X_val_standard,
            y_val=y_val_standard,

```

```

        x_test=x_test_standard,
        y_test=y_test_standard,
        epochs=hyperparameters['epochs'],
        batch_size=hyperparameters['batch_size'],
        verbose=0
    )
    val_mae = history.history['val_mae'][-1]
    test_r2 = result['R² Test']

    print(f"Val MAE: {val_mae:.4f} - Test R²: {test_r2:.4f}")

    results.append({
        "val_mae": val_mae,
        "test_r2": test_r2,
        "hyperparameters": hyperparameters,
    })

    if test_r2 > best_val_score:
        best_val_score = test_r2
        best_model_random_search = model
        best_hyperparameters = hyperparameters

    print(f"Bestes Modell nach Iteration {i+1} mit Test score:␣
↪{best_val_score:.4f} und Hyperparameter: {best_hyperparameters}")

    best_model_random_search.save(f"best_model_random_search_iterations_{i+1}.
↪h5")

    print(f"\n bestes Modell gespeichert unter:␣
↪best_model_random_search_iterations_{i+1}.h5")

    return sorted(results, key=lambda x: x["test_r2"], reverse=True)

```

```

[ ]: def r2_score(y_true, y_pred):
    ss_res = np.sum((y_true - y_pred) ** 2)
    ss_tot = np.sum((y_true - np.mean(y_true)) ** 2)
    return 1 - (ss_res / ss_tot)

```

```

[ ]: best_results = random_search(X_train_standard, y_train_standard,␣
↪X_val_standard, y_val_standard, X_test_standard, y_test_standard,␣
↪n_iterations=100)
print("Beste Ergebnisse:")
for result in best_results:
    print(f"Val MAE: {result['val_mae']:.4f} - Test R²: {result['test_r2']:.4f}␣
↪ Hyperparameter: {result['hyperparameters']}")

```

Das beste Modell noch einmal prüfen und Abspeichern testen/überprüfen

```
[46]: model_load = keras.models.load_model("../results/nn_models/
      ↪best_model_random_search_iterations_100.h5")

y_train_predictions = model_load.predict(X_train_standard, verbose=0).flatten()
y_test_predictions = model_load.predict(X_test_standard, verbose=0).flatten()

results = evaluate_predictions(y_train_standard, y_train_predictions,
      ↪y_test_standard, y_test_predictions, "Bestes Modell Random Search")
```

```
=====
Bestes Modell Random Search
=====
R² Score:  Train = 0.7925   |  Test = 0.7759
MAE:       Train = 0.3063   |  Test = 0.3146
RMSE:      Train = 0.4371   |  Test = 0.4512
=====
```

2.4 Learning-Rate-Schedule

learning-rate wird während des Trainings automatisch angebracht

KI-Prompt: Verbesserungen an Hyperparameter liegen innerhalb Schwankungen des Trainings, welche Möglichkeiten gibt es das Projekt weiter zu verbessern? > Was ist LR-Schedule?

```
[47]: from tensorflow.keras.callbacks import ReduceLROnPlateau

lr_scheduler = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=5,
    verbose=1
)
```

```
[ ]: model_1 = keras.Sequential([
    layers.Input(shape=(X_train_standard.shape[1],)),
    layers.Dense(64, activation="tanh"),
    layers.Dense(32, activation="tanh"),
    layers.Dense(1)
])

model_2 = keras.Sequential([
    layers.Input(shape=(X_train_standard.shape[1],)),
    layers.Dense(64, activation="tanh"),
    layers.Dense(32, activation="tanh"),
    layers.Dense(1)
])
```

```

model_1.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.0054),
    loss='mse',
    metrics=['mae'],
)
model_2.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.0054),
    loss='mse',
    metrics=['mae'],
)

test_mae_per_epoch_1 = []
test_mae_per_epoch_2 = []
test_R2_per_epoch_1 = []
test_R2_per_epoch_2 = []

for epoch in range(1, 501):

    model_1.fit(
        X_train_standard, y_train_standard,
        validation_data=(X_val_standard, y_val_standard),
        epochs=1,
        batch_size=32,
        verbose=0,
        shuffle=True
    )

    model_2.fit(
        X_train_standard, y_train_standard,
        validation_data=(X_val_standard, y_val_standard),
        epochs=1,
        batch_size=32,
        verbose=0,
        shuffle=True,
        callbacks=[lr_scheduler]
    )

    mse_1, mae_1 = model_1.evaluate(X_test_standard, y_test_standard, verbose=0)
    mse_2, mae_2 = model_2.evaluate(X_test_standard, y_test_standard, verbose=0)
    y_test_predictions_1 = model_1.predict(X_test_standard, verbose=0).flatten()
    y_test_predictions_2 = model_2.predict(X_test_standard, verbose=0).flatten()
    ss_res_1 = np.sum((y_test_standard - y_test_predictions_1) ** 2)
    ss_res_2 = np.sum((y_test_standard - y_test_predictions_2) ** 2)
    ss_tot = np.sum((y_test_standard - np.mean(y_test_standard)) ** 2)
    r2_score_1 = 1 - (ss_res_1 / ss_tot)
    r2_score_2 = 1 - (ss_res_2 / ss_tot)
    test_R2_per_epoch_1.append(r2_score_1)

```

```

test_R2_per_epoch_2.append(r2_score_2)
test_mae_per_epoch_1.append(mae_1)
test_mae_per_epoch_2.append(mae_2)

plt.figure(figsize=(10, 5))
plt.plot(test_mae_per_epoch_1, label="Test MAE Model 1")
plt.plot(test_mae_per_epoch_2, label="Test MAE Model 2")
plt.plot(test_R2_per_epoch_1, label="Test R2 Model 1")
plt.plot(test_R2_per_epoch_2, label="Test R2 Model 2")
plt.xlabel("Epoche")
plt.ylabel("Score")
plt.title("Test-MAE und Test-R2 pro Epoche (1-500)_2")
plt.grid(True)
plt.legend()
plt.show()

```

KI-Prompt [10]: Warum unterscheiden sich die Modelle nicht wesentlich?

Das Problem ist, dass immer nur über eine Epoche trainiert wird und daher die Lernrate nicht angepasst werden kann, daher folgende Codeverbesserung von KI:

```

[ ]: EPOCHS = 500
    BLOCK = 10

for start in range(0, EPOCHS, BLOCK):
    end = min(start + BLOCK, EPOCHS)

    history_1 = model_1.fit(
        X_train_standard, y_train_standard,
        validation_data=(X_val_standard, y_val_standard),
        epochs=end,
        initial_epoch=start,
        batch_size=32,
        verbose=0
    )

    history_2 = model_2.fit(
        X_train_standard, y_train_standard,
        validation_data=(X_val_standard, y_val_standard),
        epochs=end,
        initial_epoch=start,
        batch_size=32,
        verbose=0,
        callbacks=[lr_scheduler]
    )

    for epoch in range(start, end):

```

```

        mse1, mae1 = model_1.evaluate(X_test_standard, y_test_standard,
↪verbose=0)
        mse2, mae2 = model_2.evaluate(X_test_standard, y_test_standard,
↪verbose=0)

        y_pred1 = model_1.predict(X_test_standard, verbose=0).flatten()
        y_pred2 = model_2.predict(X_test_standard, verbose=0).flatten()

        r2_1 = 1 - np.sum((y_test_standard - y_pred1)**2) / ss_tot
        r2_2 = 1 - np.sum((y_test_standard - y_pred2)**2) / ss_tot

        test_mae_per_epoch_1.append(mae1)
        test_mae_per_epoch_2.append(mae2)
        test_R2_per_epoch_1.append(r2_1)
        test_R2_per_epoch_2.append(r2_2)

```

```

[ ]: plt.figure(figsize=(10, 5))
plt.plot(test_mae_per_epoch_1[:500], label="Test MAE kein LR-Schedule")
plt.plot(test_mae_per_epoch_2[:500], label="Test MAE mit LR-Schedule")
plt.plot(test_R2_per_epoch_1[:500], label="Test R² kein LR-Schedule")
plt.plot(test_R2_per_epoch_2[:500], label="Test R² mit LR-Schedule")
plt.xlabel("Epoche")
plt.ylabel("Score")
plt.title("Test-MAE und Test-R² pro Epoche (1-500)_2")
plt.grid(True)
plt.legend()
plt.show()

```

2.5 Training auf die wichtigsten Features

```

[49]: important_features = ["MedInc", "AveOccup", "Latitude", "Longitude"] # aus_
↪rf_feature_importance.png
# In Indizes umwandeln für Numpy Arrays
important_indices = [feature_names.index(feature) for feature in_
↪important_features]
X_train_important = X_train_standard[:, important_indices]
y_train_important = y_train_standard[:]
X_test_important = X_test_standard[:, important_indices]
y_test_important = y_test_standard[:]
X_val_important = X_val_standard[:, important_indices]
y_val_important = y_val_standard[:]

for i in range(5):

    model_name = f"NN_important_features_iteration_{i}"
    model = keras.Sequential()
    model.add(layers.Input(shape=(X_train_important.shape[1],)))

```

```

model.add(layers.Dense(64, activation="tanh"))
model.add(layers.Dense(32, activation="tanh"))
model.add(layers.Dense(1))
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.005),
    loss='mse',
    metrics=['mae'],
)
#train_and_evaluate(model, model_name, X_train_important,
↪y_train_important, X_val_important, y_val_important, X_test_important,
↪y_test_important, epochs=300, batch_size=32, verbose=0)

```

Nur das Feature mit der stärksten Korrelation: MedInc

```

[ ]: important_features = ["MedInc"] # aus rf_feature_importance.png
# In Indizes umwandeln für Numpy Arrays
important_indices = [feature_names.index(feature) for feature in
↪important_features]
X_train_important = X_train_standard[:, important_indices]
y_train_important = y_train_standard[:]
X_test_important = X_test_standard[:, important_indices]
y_test_important = y_test_standard[:]
X_val_important = X_val_standard[:, important_indices]
y_val_important = y_val_standard[:]

results = []
best_test_score = -float('inf')

for i in range(5):

    model_name = f"NN_important_features_iteration_{i}"
    model = keras.Sequential()
    model.add(layers.Input(shape=(X_train_important.shape[1],)))
    model.add(layers.Dense(64, activation="tanh"))
    model.add(layers.Dense(32, activation="tanh"))
    model.add(layers.Dense(1))
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=0.005),
        loss='mse',
        metrics=['mae'],
    )
    history, result = train_and_evaluate(model, model_name, X_train_important,
↪y_train_important, X_val_important, y_val_important, X_test_important,
↪y_test_important, epochs=300, batch_size=32, verbose=0)

    test_r2 = result['R² Test']
    print(f"Iteration {i+1} - Test R²: {test_r2:.4f}")

```



```

    if test_r2 > best_test_score:
        best_test_score = test_r2
        best_model_important_features = model

best_model_important_features.
    ↪ save(f"best_model_important_features_iterations_{i+1}.h5")
print(f"\n bestes Modell gespeichert unter:␣
    ↪ best_model_important_features_iterations_{i+1}.h5")

```

Nochmal die besten vier Features aber diesmal speichern wir das Modell ;)

```

[ ]: important_features = ["MedInc", "AveOccup", "Latitude", "Longitude"] # aus␣
    ↪ rf_feature_importance.png
# In Indizes umwandeln für Numpy Arrays
important_indices = [feature_names.index(feature) for feature in␣
    ↪ important_features]
X_train_important = X_train_standard[:, important_indices]
y_train_important = y_train_standard[:]
X_test_important = X_test_standard[:, important_indices]
y_test_important = y_test_standard[:]
X_val_important = X_val_standard[:, important_indices]
y_val_important = y_val_standard[:]

results = []
best_test_score = -float('inf')

for i in range(5):

    model_name = f"NN_4_important_features_iteration_{i}"
    model = keras.Sequential()
    model.add(layers.Input(shape=(X_train_important.shape[1],)))
    model.add(layers.Dense(64, activation="tanh"))
    model.add(layers.Dense(32, activation="tanh"))
    model.add(layers.Dense(1))
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=0.005),
        loss='mse',
        metrics=['mae'],
    )
    history, result = train_and_evaluate(model, model_name, X_train_important,␣
    ↪ y_train_important, X_val_important, y_val_important, X_test_important,␣
    ↪ y_test_important, epochs=300, batch_size=32, verbose=0)

    test_r2 = result['R² Test']
    print(f"Iteration {i+1} - Test R²: {test_r2:.4f}")
    if test_r2 > best_test_score:
        best_test_score = test_r2

```

```

        best_model_4_important_features = model

best_model_4_important_features.
    ↳save(f"best_model_4_important_features_iterations_{i+1}.h5")
print(f"\n bestes Modell gespeichert unter:␣
    ↳best_model_4_important_features_iterations_{i+1}.h5")

```

```

[52]: fig, axes = plt.subplots(1, 2, figsize=(14, 6))

model_important_features = keras.models.load_model("../results/nn_models/
    ↳best_model_4_important_features_iterations_5.h5")
model_best_random_search = keras.models.load_model("../results/nn_models/
    ↳best_model_random_search_iterations_100.h5")

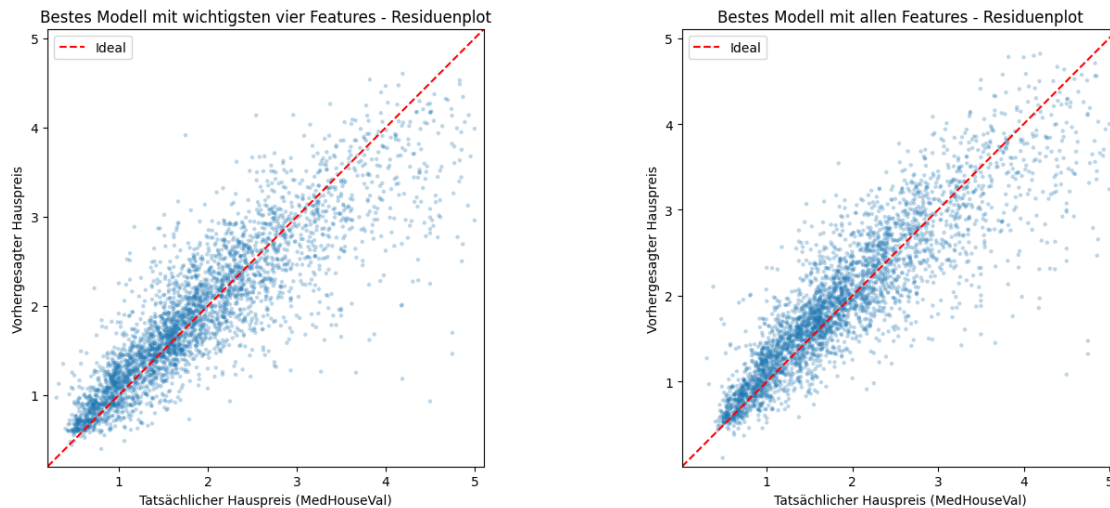
y_pred_important_features = model_important_features.predict(X_test_important,␣
    ↳verbose=0).flatten()
plot_predicted_vs_actual(
    y_test_important,
    y_pred_important_features,
    title="Bestes Modell mit wichtigsten vier Features - Residuenplot",
    ax=axes[0]
)

y_pred_best_model = model_best_random_search.predict(X_test_standard,␣
    ↳verbose=0).flatten()
plot_predicted_vs_actual(
    y_test_standard,
    y_pred_best_model,
    title="Bestes Modell mit allen Features - Residuenplot",
    ax=axes[1]
)

fig.suptitle("Vergleich alle und vier wichtigsten Features", fontsize=14)
fig.tight_layout()
#save_fig(fig, 'nn_4_vs_8_Features')
plt.show()

```

Vergleich alle und vier wichtigsten Features



wichtigsten 5 Features testen -> eventuell noch mehr Informationen wie rauschen

```
[ ]: important_features = ["MedInc", "AveOccup", "Latitude", "Longitude",
    ↪ "HouseAge"] # aus rf_feature_importance.png
# In Indizes umwandeln für Numpy Arrays
important_indices = [feature_names.index(feature) for feature in
    ↪ important_features]
X_train_important = X_train_standard[:, important_indices]
y_train_important = y_train_standard[:]
X_test_important = X_test_standard[:, important_indices]
y_test_important = y_test_standard[:]
X_val_important = X_val_standard[:, important_indices]
y_val_important = y_val_standard[:]

results = []
best_test_score = -float('inf')

for i in range(5):

    model_name = f"NN_5_important_features_iteration_{i}"
    model = keras.Sequential()
    model.add(layers.Input(shape=(X_train_important.shape[1],)))
    model.add(layers.Dense(64, activation="tanh"))
    model.add(layers.Dense(32, activation="tanh"))
    model.add(layers.Dense(1))
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=0.005),
        loss='mse',
        metrics=['mae'],
```

```

    )
    history, result = train_and_evaluate(model, model_name, X_train_important,
    ↪ y_train_important, X_val_important, y_val_important, X_test_important,
    ↪ y_test_important, epochs=300, batch_size=32, verbose=0)

    test_r2 = result['R2 Test']
    print(f"Iteration {i+1} - Test R2: {test_r2:.4f}")
    if test_r2 > best_test_score:
        best_test_score = test_r2
        best_model_5_important_features = model

best_model_5_important_features.
    ↪ save(f"best_model_5_important_features_iterations_{i+1}.h5")
print(f"\n bestes Modell gespeichert unter:
    ↪ best_model_5_important_features_iterations_{i+1}.h5")

```

2.6 Analyse des Modells in Hinblick auf Verbesserung der Hyperparameter

2.6.1 Residuen Histogramm

```

[78]: important_features = ["MedInc", "AveOccup", "Latitude", "Longitude"] # aus
    ↪ rf_feature_importance.png
# In Indizes umwandeln für Numpy Arrays
important_indices = [feature_names.index(feature) for feature in
    ↪ important_features]
X_test_important = X_test_standard[:, important_indices]

model_important_features = keras.models.load_model("../results/nn_models/
    ↪ best_model_4_important_features_iterations_5.h5")
model_best_random_search = keras.models.load_model("../results/nn_models/
    ↪ best_model_random_search_iterations_100.h5")

y_pred_important_features = model_important_features.predict(X_test_important,
    ↪ verbose=0).flatten()
y_pred_best_model = model_best_random_search.predict(X_test_standard,
    ↪ verbose=0).flatten()

#res_4 = y_test_important - y_pred_important_features
#res_all = y_test_standard - y_pred_best_model

#mu_4, std_4 = np.mean(res_4), np.std(res_4)
#mu_all, std_all = np.mean(res_all), np.std(res_all)

plt.rcParams.update({
    "font.size": 16,
    "axes.labelsize": 18,
    "axes.titlesize": 20,

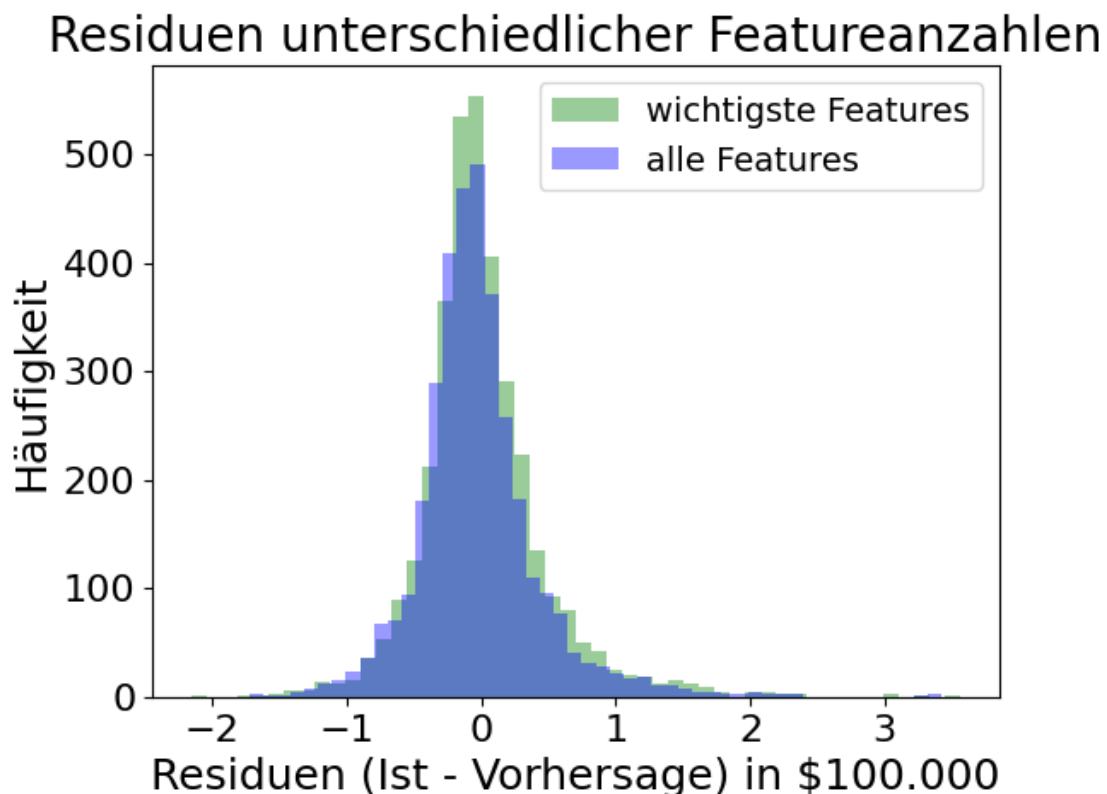
```

```

    "legend.fontsize": 14,
    "xtick.labelsize": 16,
    "ytick.labelsize": 16
})

plt.hist(y_test_important - y_pred_important_features, bins=50, alpha=0.4,
        color='g', label='wichtigste Features')
plt.hist(y_test_standard - y_pred_best_model, bins=50, alpha=0.4, color='b',
        label='alle Features')
plt.legend(loc='upper right')
plt.xlabel('Residuen (Ist - Vorhersage) in $100.000')
plt.ylabel('Häufigkeit')
plt.title('Residuen unterschiedlicher Featureanzahlen')
#save_fig(plt, 'nn_residuals_histogramme_vergleich_featureanzahl_2')
plt.show()

```



Analyse: - liegen symmetrisch um die Null: kein systematischer Fehler im Netz - Ausreißer gehen eher in die Positive Richtung, d.h. es gibt stärkere Fälle, bei denen der tatsächliche Preis höher als der vorhergesagte Preis ist - über die Breite beider Verteilungen lässt sich dies schlecht händisch einschätzen. Im Bereich um die Null ist grün höher, was bei der selben Sample zahl eher für eine schmalere Verteilung spricht (kann aber auch durch die Form etc. nicht schmaler sein)

2.6.2 Residuen vs. Feature Plot

Für die wichtigsten 4 Features

```
[70]: residuals = y_pred_important_features - y_test_important
n_features = X_test_important.shape[1]
cols = 2
rows = int(np.ceil(n_features / cols))
fig, axes = plt.subplots(rows, cols, figsize=(18, 5 * rows))
axes = axes.flatten()

feature_zu_index = {feature: idx for idx, feature in
    ↪enumerate(important_features)}

for i, feature in enumerate(important_features):
    idx = feature_zu_index[feature]
    ax = axes[i]
    ax.scatter(X_test_important[:, idx], residuals, alpha=0.5)
    ax.axhline(0, color='red', linestyle='--')
    ax.set_xlabel(feature)
    ax.set_ylabel('Residuen')
    ax.set_title(f'Residuen vs {feature}')

for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

for i, feature in enumerate(important_features):
    idx = feature_zu_index[feature]
    ax = axes[i]

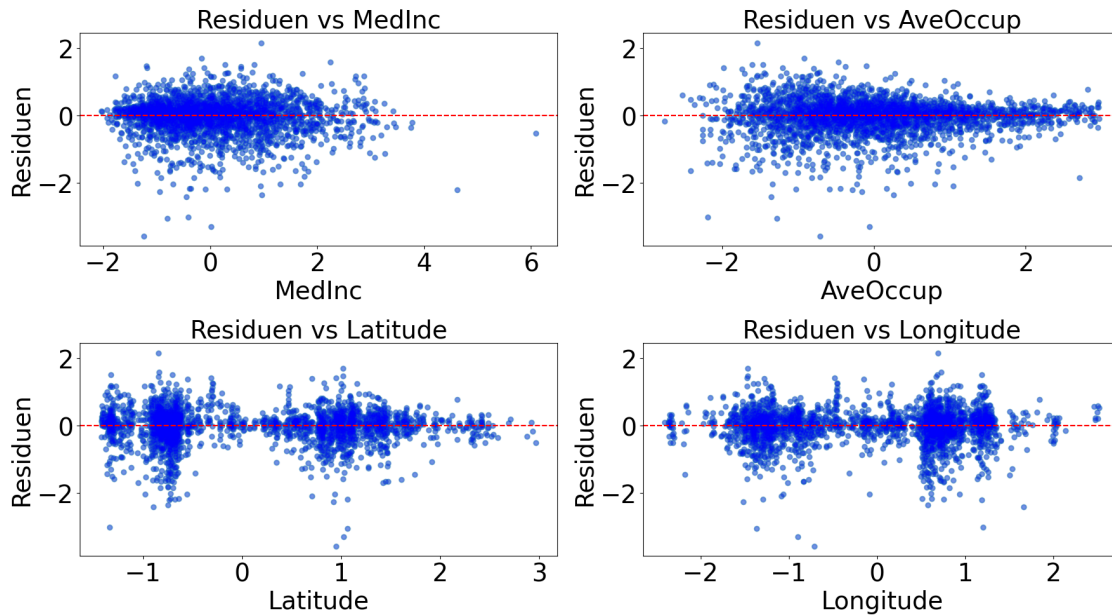
    ax.scatter(
        X_test_important[:, idx],
        residuals,
        color='blue',
        alpha=0.2,
        s=20
    )

    ax.axhline(0, color='red', linestyle='--')

    ax.set_xlabel(feature, fontsize=28)
    ax.set_ylabel('Residuen', fontsize=28)
    ax.set_title(f'Residuen vs {feature}', fontsize=28)
    ax.tick_params(axis='both', labelsize=28)
```

```
fig.tight_layout()
#save_fig(fig, 'nn_residuals_vs_4_wichtigste_features_2')
plt.show
```

[70]: <function matplotlib.pyplot.show(close=None, block=None)>



Interpretation des Ergebnisses mithilfe von [6]:

- keine U oder S-Form: Zusammenhänge korrekt gelernt
- keine Cluster außer bei Latitude und Longitude -> Feature Engineering
- kein Bias -> alle um Null herum
- Latitude und Longitude erkennbar weniger relevant als die anderen beiden Features

2.7 Ensemble der besten Modelle

- prüfen, ob die Modelle unterschiedliche Fehler machen oder identische Fehler
- bessere Generalisierung durch die hohe Schwankung bei erneuter Ausführung
- Outliers glätten
- verschiedene Aktivierungsfunktionen und Hyperparameter, daher gehe ich davon aus, dass Modelle leichte Unterschiede haben und ein Ensemble sinnvoll sein kann

```
[55]: model_ensemble_1 = keras.models.load_model("../results/nn_models/
↳best_model_random_search_iterations_100.h5")
model_ensemble_2 = keras.models.load_model("../results/nn_models/
↳best_model_4_important_features_iterations_5.h5")
model_ensemble_3 = keras.models.load_model("../results/nn_models/
↳best_model_5_important_features_iterations_5.h5")
```

```

model_ensemble_4 = keras.models.load_model("../results/nn_models/
↳best_model_random_search_iteration_20.h5")

model_1_pred = model_ensemble_1.predict(X_test_standard, verbose=0).flatten()

important_features = ["MedInc", "AveOccup", "Latitude", "Longitude"]
important_indices = [feature_names.index(feature) for feature in
↳important_features]
X_test_4_important = X_test_standard[:, important_indices]
model_2_pred = model_ensemble_2.predict(X_test_4_important, verbose=0).flatten()

important_features = ["MedInc", "AveOccup", "Latitude", "Longitude", "HouseAge"]
important_indices = [feature_names.index(feature) for feature in
↳important_features]
X_test_5_important = X_test_standard[:, important_indices]
model_3_pred = model_ensemble_3.predict(X_test_5_important, verbose=0).flatten()

model_4_pred = model_ensemble_4.predict(X_test_standard, verbose=0).flatten()

y_ensemble_predictions = np.mean([model_1_pred, model_2_pred, model_3_pred,
↳model_4_pred], axis=0);
evaluate_test_predictions(y_test_standard, y_ensemble_predictions, "Ensemble
↳Modell")

```

```

=====
Ensemble Modell
=====
R2 Score:  Test = 0.7946
MAE:        Test = 0.2944
RMSE:       Test = 0.4321
=====

```

Eindeutig das beste Ergebnis bis jetzt, konnte durch Hyperparameter Verbesserung nicht erreicht werden: - Modelle sind nicht redundant (verschiedene Feature subsets und Hyperparameter) - Fehler der Modelle sind nicht perfekt korreliert -> Anderes Modell macht andere Fehler - Ensemble glättet ausreißer? -> residuen plot und histogramm anschauen...

```

[56]: fig, axes = plt.subplots(1, 2, figsize=(14, 6))

model_best_random_search = keras.models.load_model("../results/nn_models/
↳best_model_random_search_iterations_100.h5")

plot_predicted_vs_actual(
    y_test_standard,
    y_ensemble_predictions,
    title="Ensemble Modell - Residuenplot",

```



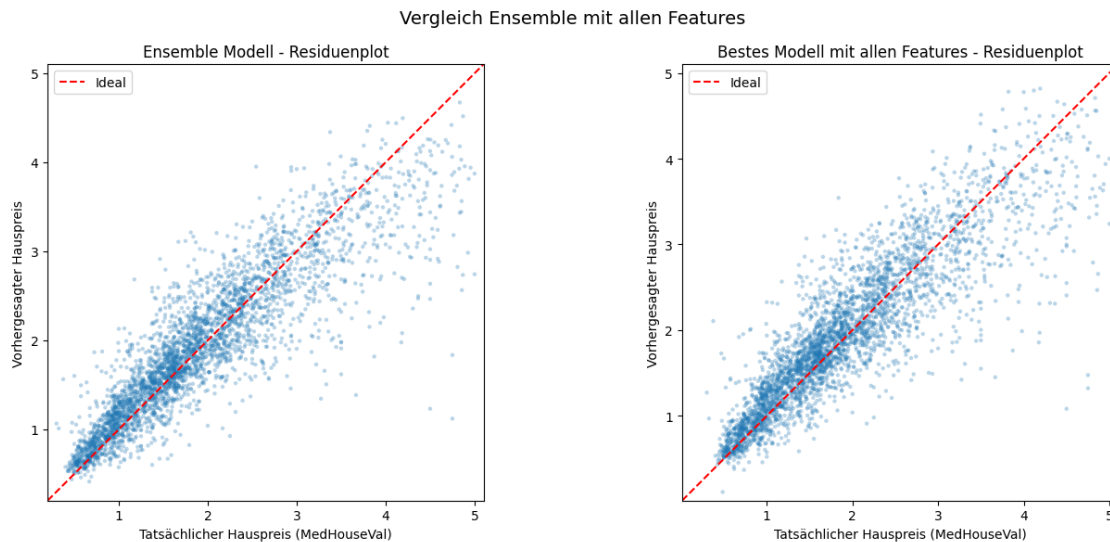
```

    ax=axes[0]
)

y_pred_best_model = model_best_random_search.predict(X_test_standard,
↳ verbose=0).flatten()
plot_predicted_vs_actual(
    y_test_standard,
    y_pred_best_model,
    title="Bestes Modell mit allen Features - Residuenplot",
    ax=axes[1]
)

fig.suptitle("Vergleich Ensemble mit allen Features", fontsize=14)
fig.tight_layout()
#save_fig(fig, 'nn_Ensemble_vs_8_Features_2')
plt.show()

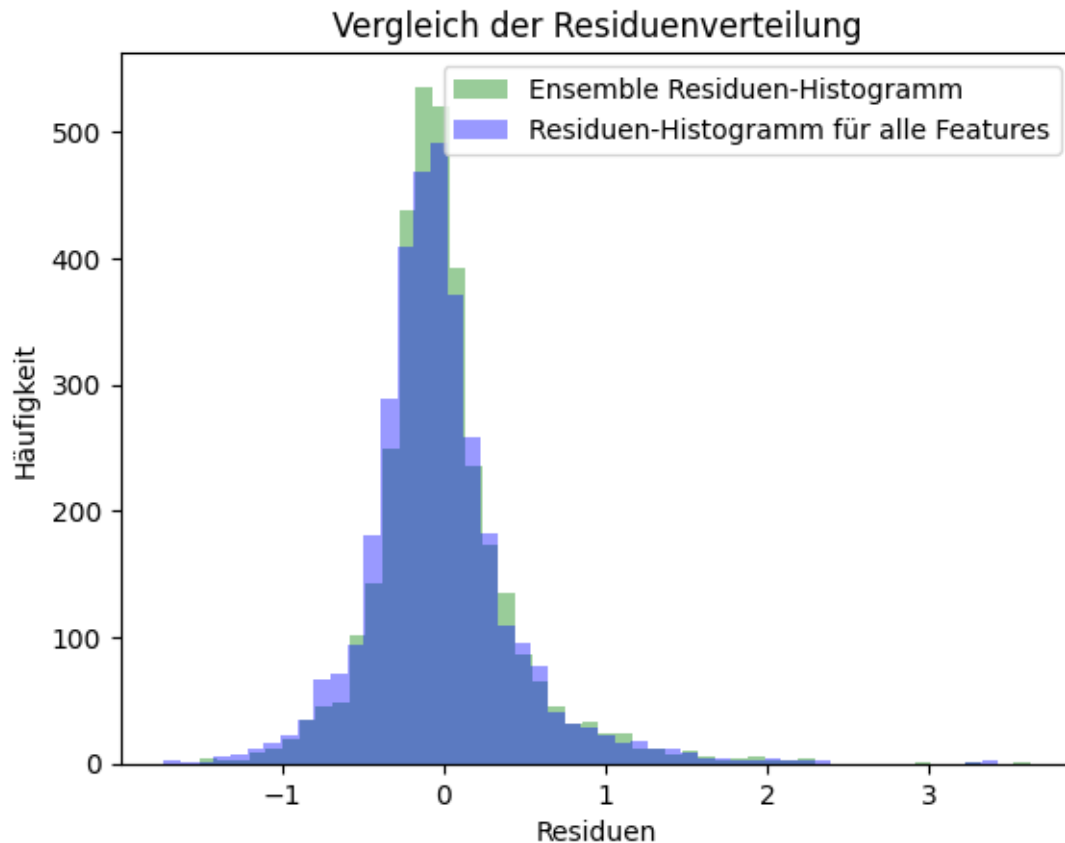
```



```

[57]: plt.hist(y_test_standard - y_ensemble_predictions, bins=50, alpha=0.4,
↳ color='g', label='Ensemble Residuen-Histogramm')
plt.hist(y_test_standard - y_pred_best_model, bins=50, alpha=0.4, color='b',
↳ label='Residuen-Histogramm für alle Features')
plt.legend()
plt.xlabel('Residuen')
plt.ylabel('Häufigkeit')
plt.title('Vergleich der Residuenverteilung')
#save_fig(plt,
↳ 'nn_residuals_histogramme_Ensemble_vs_best_model_random_search_2')
plt.show()

```



Verschiedene Modellkombinationen testen

```
[58]: from itertools import combinations

models_pred_for_ensemble = [model_1_pred, model_2_pred, model_3_pred,
                             ↪model_4_pred]
names = ["Model 1 (8F-100iter)", "Model 2 (4F)", "Model 3 (5F)", "Model 4_
                             ↪(8F-20iter)"]

for r in range(2, len(models_pred_for_ensemble) + 1):
    print(f"\nEnsemble aus {r} Modellen:")
    for combo in combinations(range(len(models_pred_for_ensemble)), r):

        preds = np.mean([models_pred_for_ensemble[i] for i in combo], axis=0)
        combo_names = " + ".join(names[i] for i in combo)
        print(f"Ensemble: {combo_names}")
        evaluate_test_predictions(y_test_standard, preds, f"Ensemble:
        ↪{combo_names}")
```

Ensemble aus 2 Modellen:

Ensemble: Model 1 (8F-100iter) + Model 2 (4F)

```
=====
Ensemble: Model 1 (8F-100iter) + Model 2 (4F)
=====
R2 Score:  Test = 0.7892
MAE:       Test = 0.3000
RMSE:      Test = 0.4377
=====
```

Ensemble: Model 1 (8F-100iter) + Model 3 (5F)

```
=====
Ensemble: Model 1 (8F-100iter) + Model 3 (5F)
=====
R2 Score:  Test = 0.7879
MAE:       Test = 0.3013
RMSE:      Test = 0.4390
=====
```

Ensemble: Model 1 (8F-100iter) + Model 4 (8F-20iter)

```
=====
Ensemble: Model 1 (8F-100iter) + Model 4 (8F-20iter)
=====
R2 Score:  Test = 0.7723
MAE:       Test = 0.3138
RMSE:      Test = 0.4549
=====
```

Ensemble: Model 2 (4F) + Model 3 (5F)

```
=====
Ensemble: Model 2 (4F) + Model 3 (5F)
=====
R2 Score:  Test = 0.7797
MAE:       Test = 0.3057
RMSE:      Test = 0.4474
=====
```

Ensemble: Model 2 (4F) + Model 4 (8F-20iter)

```
=====
Ensemble: Model 2 (4F) + Model 4 (8F-20iter)
=====
R2 Score:  Test = 0.7795
MAE:       Test = 0.3057
RMSE:      Test = 0.4477
=====
```

Ensemble: Model 3 (5F) + Model 4 (8F-20iter)

```
=====
```

```

Ensemble: Model 3 (5F) + Model 4 (8F-20iter)
=====
R2 Score:  Test = 0.7754
MAE:        Test = 0.3086
RMSE:       Test = 0.4518
=====

Ensemble aus 3 Modellen:
Ensemble: Model 1 (8F-100iter) + Model 2 (4F) + Model 3 (5F)

=====
Ensemble: Model 1 (8F-100iter) + Model 2 (4F) + Model 3 (5F)
=====
R2 Score:  Test = 0.7921
MAE:        Test = 0.2965
RMSE:       Test = 0.4346
=====
Ensemble: Model 1 (8F-100iter) + Model 2 (4F) + Model 4 (8F-20iter)

=====
Ensemble: Model 1 (8F-100iter) + Model 2 (4F) + Model 4 (8F-20iter)
=====
R2 Score:  Test = 0.7901
MAE:        Test = 0.2989
RMSE:       Test = 0.4368
=====
Ensemble: Model 1 (8F-100iter) + Model 3 (5F) + Model 4 (8F-20iter)

=====
Ensemble: Model 1 (8F-100iter) + Model 3 (5F) + Model 4 (8F-20iter)
=====
R2 Score:  Test = 0.7883
MAE:        Test = 0.3002
RMSE:       Test = 0.4386
=====
Ensemble: Model 2 (4F) + Model 3 (5F) + Model 4 (8F-20iter)

=====
Ensemble: Model 2 (4F) + Model 3 (5F) + Model 4 (8F-20iter)
=====
R2 Score:  Test = 0.7893
MAE:        Test = 0.2978
RMSE:       Test = 0.4376
=====

Ensemble aus 4 Modellen:
Ensemble: Model 1 (8F-100iter) + Model 2 (4F) + Model 3 (5F) + Model 4
(8F-20iter)

```

```
=====
Ensemble: Model 1 (8F-100iter) + Model 2 (4F) + Model 3 (5F) + Model 4
(8F-20iter)
=====
```

```
R2 Score:  Test = 0.7946
MAE:       Test = 0.2944
RMSE:      Test = 0.4321
=====
```

Je mehr Modelle im Ensemble berücksichtigt werden, desto besser wird der Test-Score. Der Score ist besser, wenn man Modelle mit unterschiedlicher Featureanzahl kombiniert.

Nächster Schritt, mehr Modelle erzeugen und diese in das Ensemble einfließen lassen.

```
[ ]: important_features = ["MedInc", "AveOccup", "Latitude"] # aus rf_feature_importance.png
      ↪ rf_feature_importance.png
# In Indizes umwandeln für Numpy Arrays
important_indices = [feature_names.index(feature) for feature in
      ↪ important_features]
X_train_important = X_train_standard[:, important_indices]
y_train_important = y_train_standard[:]
X_test_important = X_test_standard[:, important_indices]
y_test_important = y_test_standard[:]
X_val_important = X_val_standard[:, important_indices]
y_val_important = y_val_standard[:]

results = []
best_test_score = -float('inf')

for i in range(5):

    model_name = f"NN_3_important_features_iteration_{i}"
    model = keras.Sequential()
    model.add(layers.Input(shape=(X_train_important.shape[1],)))
    model.add(layers.Dense(64, activation="tanh"))
    model.add(layers.Dense(32, activation="tanh"))
    model.add(layers.Dense(1))
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=0.005),
        loss='mse',
        metrics=['mae'],
    )
    history, result = train_and_evaluate(model, model_name, X_train_important,
    ↪ y_train_important, X_val_important, y_val_important, X_test_important,
    ↪ y_test_important, epochs=300, batch_size=32, verbose=0)

    test_r2 = result['R2 Test']
```

```

print(f"Iteration {i+1} - Test R²: {test_r2:.4f}")
if test_r2 > best_test_score:
    best_test_score = test_r2
    best_model_3_important_features = model

best_model_3_important_features.
↪save(f"best_model_3_important_features_iterations_{i+1}.h5")
print(f"\n bestes Modell gespeichert unter:␣
↪best_model_3_important_features_iterations_{i+1}.h5")

```

```

[ ]: important_features = ["MedInc", "AveOccup"] # aus rf_feauture_importance.png
# In Indizes umwandeln für Numpy Arrays
important_indices = [feature_names.index(feature) for feature in␣
↪important_features]
X_train_important = X_train_standard[:, important_indices]
y_train_important = y_train_standard[:]
X_test_important = X_test_standard[:, important_indices]
y_test_important = y_test_standard[:]
X_val_important = X_val_standard[:, important_indices]
y_val_important = y_val_standard[:]

results = []
best_test_score = -float('inf')

for i in range(5):

    model_name = f"NN_2_important_features_iteration_{i}"
    model = keras.Sequential()
    model.add(layers.Input(shape=(X_train_important.shape[1],)))
    model.add(layers.Dense(64, activation="tanh"))
    model.add(layers.Dense(32, activation="tanh"))
    model.add(layers.Dense(1))
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=0.005),
        loss='mse',
        metrics=['mae'],
    )
    history, result = train_and_evaluate(model, model_name, X_train_important,␣
↪y_train_important, X_val_important, y_val_important, X_test_important,␣
↪y_test_important, epochs=300, batch_size=32, verbose=0)

    test_r2 = result['R² Test']
    print(f"Iteration {i+1} - Test R²: {test_r2:.4f}")
    if test_r2 > best_test_score:
        best_test_score = test_r2
        best_model_2_important_features = model

```

```

best_model_2_important_features.
    ↪save(f"best_model_2_important_features_iterations_{i+1}.h5")
print(f"\n bestes Modell gespeichert unter:␣
    ↪best_model_2_important_features_iterations_{i+1}.h5")

```

```

[ ]: important_features = ["MedInc", "AveOccup", "Latitude", "Longitude",␣
    ↪"HouseAge", "AveRooms"] # aus rf_feauture_importance.png
# In Indizes umwandeln für Numpy Arrays
important_indices = [feature_names.index(feature) for feature in␣
    ↪important_features]
X_train_important = X_train_standard[:, important_indices]
y_train_important = y_train_standard[:]
X_test_important = X_test_standard[:, important_indices]
y_test_important = y_test_standard[:]
X_val_important = X_val_standard[:, important_indices]
y_val_important = y_val_standard[:]

results = []
best_test_score = -float('inf')

for i in range(5):

    model_name = f"NN_6_important_features_iteration_{i}"
    model = keras.Sequential()
    model.add(layers.Input(shape=(X_train_important.shape[1],)))
    model.add(layers.Dense(64, activation="tanh"))
    model.add(layers.Dense(32, activation="tanh"))
    model.add(layers.Dense(1))
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=0.005),
        loss='mse',
        metrics=['mae'],
    )
    history, result = train_and_evaluate(model, model_name, X_train_important,␣
    ↪y_train_important, X_val_important, y_val_important, X_test_important,␣
    ↪y_test_important, epochs=300, batch_size=32, verbose=0)

    test_r2 = result['R² Test']
    print(f"Iteration {i+1} - Test R²: {test_r2:.4f}")
    if test_r2 > best_test_score:
        best_test_score = test_r2
        best_model_6_important_features = model

best_model_6_important_features.
    ↪save(f"best_model_6_important_features_iterations_{i+1}.h5")
print(f"\n bestes Modell gespeichert unter:␣
    ↪best_model_6_important_features_iterations_{i+1}.h5")

```

```

[59]: model_ensemble_1 = keras.models.load_model("../results/nn_models/
      ↪best_model_random_search_iterations_100.h5")
model_ensemble_2 = keras.models.load_model("../results/nn_models/
      ↪best_model_4_important_features_iterations_5.h5")
model_ensemble_3 = keras.models.load_model("../results/nn_models/
      ↪best_model_5_important_features_iterations_5.h5")
model_ensemble_4 = keras.models.load_model("../results/nn_models/
      ↪best_model_random_search_iteration_20.h5")
#model_ensemble_5 = keras.models.load_model("../results/nn_models/
      ↪best_model_3_important_features_iterations_5.h5")
#model_ensemble_6 = keras.models.load_model("../results/nn_models/
      ↪best_model_2_important_features_iterations_5.h5")
model_ensemble_7 = keras.models.load_model("../results/nn_models/
      ↪best_model_6_important_features_iterations_5.h5")

model_1_pred = model_ensemble_1.predict(X_test_standard, verbose=0).flatten()

important_features = ["MedInc", "AveOccup", "Latitude", "Longitude"]
important_indices = [feature_names.index(feature) for feature in
      ↪important_features]
X_test_4_important = X_test_standard[:, important_indices]
model_2_pred = model_ensemble_2.predict(X_test_4_important, verbose=0).flatten()

important_features = ["MedInc", "AveOccup", "Latitude", "Longitude", "HouseAge"]
important_indices = [feature_names.index(feature) for feature in
      ↪important_features]
X_test_5_important = X_test_standard[:, important_indices]
model_3_pred = model_ensemble_3.predict(X_test_5_important, verbose=0).flatten()

model_4_pred = model_ensemble_4.predict(X_test_standard, verbose=0).flatten()

#important_features = ["MedInc", "AveOccup", "Latitude"]
#important_indices = [feature_names.index(feature) for feature in
      ↪important_features]
#X_test_3_important = X_test_standard[:, important_indices]
#model_5_pred = model_ensemble_5.predict(X_test_3_important, verbose=0).
      ↪flatten()

#important_features = ["MedInc", "AveOccup"]
#important_indices = [feature_names.index(feature) for feature in
      ↪important_features]
#X_test_2_important = X_test_standard[:, important_indices]
#model_6_pred = model_ensemble_6.predict(X_test_2_important, verbose=0).
      ↪flatten()

```



```

important_features = ["MedInc", "AveOccup", "Latitude", "Longitude",
    ↪ "HouseAge", "AveRooms"]
important_indices = [feature_names.index(feature) for feature in
    ↪ important_features]
X_test_6_important = X_test_standard[:, important_indices]
model_7_pred = model_ensemble_7.predict(X_test_6_important, verbose=0).flatten()

y_ensemble_predictions = np.mean([model_1_pred, model_2_pred, model_3_pred,
    ↪ model_4_pred, model_7_pred], axis=0);
evaluate_test_predictions(y_test_standard, y_ensemble_predictions, "Ensemble_
    ↪ Modell")

```

```

=====
Ensemble Modell
=====
R² Score:  Test = 0.7983
MAE:       Test = 0.2891
RMSE:      Test = 0.4282
=====

```

Score besser, wenn man das 2_feature Modell raus nimmt. Score noch besser, wenn man das 3_feature Modell raus nimmt

Idee: Mehrere Modelle mit verschiedenen Hyperparametern auf die vier wichtigsten Features trainieren und dann als Ensemble verknüpfen

```

[ ]: def random_search_save_best_five(X_train_standard, y_train_standard,
    ↪ X_val_standard, y_val_standard, x_test_standard, y_test_standard,
    ↪ n_iterations=25):
    results = []

    top_models = []

    input_dim = X_train_standard.shape[1]

    for i in range(n_iterations):
        hyperparameters = sample_parameters()
        print(f"Iteration {i+1}/{n_iterations} - Hyperparameter:
            ↪ {hyperparameters}")

        model = build_model(hyperparameters, input_dim)

        history, result = train_and_evaluate(
            model,
            f"RandomSearch_{i+1}",
            x_train=X_train_standard,
            y_train=y_train_standard,

```

```

        x_val=X_val_standard,
        y_val=y_val_standard,
        x_test=x_test_standard,
        y_test=y_test_standard,
        epochs=hyperparameters['epochs'],
        batch_size=hyperparameters['batch_size'],
        verbose=0
    )
    val_mae = history.history['val_mae'][-1]
    test_r2 = result['R² Test']

    print(f"Val MAE: {val_mae:.4f} - Test R²: {test_r2:.4f}")

    results.append({
        "val_mae": val_mae,
        "test_r2": test_r2,
        "hyperparameters": hyperparameters,
    })

    top_models.append((test_r2, model, hyperparameters))
    top_models = sorted(top_models, key=lambda x: x[0], reverse=True)[:5]

    print(f"aktuell bestes Modell nach Iteration {i+1} mit Test score:␣
    ↳{top_models[0][0]:.4f} und Hyperparameter: {top_models[0][2]}")

    for rank, (score, model, hyperparameters) in enumerate(top_models, start=1):
        filename = f"best_model_random_search_rank_{rank}_score_{score:.4f}.h5"
        model.save(filename)
        print(f"Modell mit Rang {rank} gespeichert unter: {filename}")

    return sorted(results, key=lambda x: x["test_r2"], reverse=True)

```

```

[ ]: important_features = ["MedInc", "AveOccup", "Latitude", "Longitude"]
important_indices = [feature_names.index(feature) for feature in␣
    ↳important_features]
X_train_important = X_train_standard[:, important_indices]
y_train_important = y_train_standard[:]
X_test_important = X_test_standard[:, important_indices]
y_test_important = y_test_standard[:]
X_val_important = X_val_standard[:, important_indices]
y_val_important = y_val_standard[:]

best_results = random_search_save_best_five(X_train_important,␣
    ↳y_train_important, X_val_important, y_val_important, X_test_important,␣
    ↳y_test_important, n_iterations=25)
print("Beste Ergebnisse:")

```

```

for result in best_results:
    print(f"Val MAE: {result['val_mae']:.4f} - Test R²: {result['test_r2']:.4f}␣
    ↳ Hyperparameter: {result['hyperparameters']}")

```

```

[60]: model_ensemble_1 = keras.models.load_model("../results/nn_models/
    ↳best_model_random_search_rank_1_score_0.7796.h5")
model_ensemble_2 = keras.models.load_model("../results/nn_models/
    ↳best_model_random_search_rank_2_score_0.7656.h5")
model_ensemble_3 = keras.models.load_model("../results/nn_models/
    ↳best_model_random_search_rank_3_score_0.7644.h5")
model_ensemble_4 = keras.models.load_model("../results/nn_models/
    ↳best_model_random_search_rank_4_score_0.7606.h5")
model_ensemble_5 = keras.models.load_model("../results/nn_models/
    ↳best_model_random_search_rank_5_score_0.7573.h5")

important_features = ["MedInc", "AveOccup", "Latitude", "Longitude"]
important_indices = [feature_names.index(feature) for feature in␣
    ↳important_features]
X_test_4_important = X_test_standard[:, important_indices]
model_1_pred = model_ensemble_1.predict(X_test_4_important, verbose=0).flatten()
model_2_pred = model_ensemble_2.predict(X_test_4_important, verbose=0).flatten()
model_3_pred = model_ensemble_3.predict(X_test_4_important, verbose=0).flatten()
model_4_pred = model_ensemble_4.predict(X_test_4_important, verbose=0).flatten()
model_5_pred = model_ensemble_5.predict(X_test_4_important, verbose=0).flatten()

y_ensemble_predictions = np.mean([model_1_pred, model_2_pred, model_3_pred,␣
    ↳model_4_pred, model_5_pred], axis=0);
evaluate_test_predictions(y_test_standard, y_ensemble_predictions, "Ensemble␣
    ↳Modell mit verschiedenen Hyperparametern")

```

```

=====
Ensemble Modell mit verschiedenen Hyperparametern
=====
R² Score:  Test = 0.7861
MAE:       Test = 0.3074
RMSE:      Test = 0.4409
=====

```

3 Zusammenfassung 05_Neuronal_Network_Felix:

3.1 Hyperparametertraining:

- Lernrate: Um die 0,05 ist das Training am stabilsten
- Aktivierungsfunktion: Liegen innerhalb der Varianz bei erneutem Ausführen. Jede Ak-

tivierungsfunktion die behandelt worden ist, sollte mit den standardisierten Daten arbeiten

- Batch-Size: zwischen 16 und 128 hat die Batch-Size einen unwesentlichen Einfluss auf das gelernte Modell
- Epochenanzahl: 100 Epochen sind zu wenig für eine Konvergenz, 300 bis 500 Epochen sind ideal. Ab 500 Epochen verstärkt sich das Overfitting => Änderungen in Hyperparameter haben nur noch sehr geringe Abhängigkeiten zur Modellqualität, da die Varianz des Modellrainings höher ist

3.2 Random Search:

Mittels Random Search konnte der R^2 Score auf den Testdaten verbessert werden, in dem zufällige Parameterkombinationen getestet worden sind. Mithilfe der Arbeit von Teamkollege Kolja sollte erneut Random Search mit allen Hyperparametern durchgeführt werden

3.3 LR-Schedule:

Bringt keine Verbesserung

3.4 Featurereduktion:

Mit den vier wichtigsten Features kann der Test Score reproduziert werden. Die anderen vier Features enthalten keine neuen Informationen bzw. werden diese durch Rauschen überdeckt. Weniger als vier Features ist nicht ratsam, da wichtige Informationen verloren gehen.

3.5 Ensemble:

Durch Bildung eines Mittelwerts konnte die Vorhersage deutlich verbessert werden. Je nachdem wie viele Features ausgewählt sind macht das Modell andere Vorhersagen, wodurch eine Mittelung die Vorhersage stabilisieren kann. Sobald weniger als vier Modelle für das Averaging verwendet werden sinkt der Score des Ensemble.